STOQUIFY · ACCOUNTING REPORTS

Accounting Financial Reporting Suite — Execution Prompt

Ledger-backed Profit & Loss, management EBITDA, and Balance Sheet execution contract

Generated 2026-08-02

Internal project documentation

Refined Professional Prompt

```
Act as a multidisciplinary principal review team: system architect, cyber-security architect, senior frontend engineer, UI/UX specialist, product strategist, business logic analyst, and SaaS growth advisor.

Project:
AqStoqFlow / Kontava platform.

Workspace:
`E:\ohada saas\Focused projects\stoquify`

Domain:
Accounting financial reporting, management performance reporting, and OHADA-aware presentation boundaries.

Mission:
Create the smallest high-value professional accounting reporting suite that closes the current gap beyond the trial balance. Add a posted-ledger Financial Reports hub, Profit & Loss statement, a clearly labelled management EBITDA measure with transparent reconciliation and classification coverage, and a Balance Sheet with an explicit balance check. Make the suite modern, responsive, useful for recurring management review, and trustworthy enough to increase product adoption without presenting internal read models as certified OHADA statutory filings.

Domain lens:
Act as a senior enterprise accounting reporting architecture team:

- Senior enterprise software architect: preserve accounting service ownership, the posted-ledger source of truth, dependency order, platform modularity, and the existing action/service contracts.
- Structural UI/UX design expert: build period-first, role-aware, compact and responsive financial-reporting surfaces only after service-owned statement read models and classification contracts exist.
- Cybersecurity and RBAC specialist: enforce tenant isolation, `accounting.reports.read`, accounting module entitlement, server-side authorization, audit-safe responses, redaction, and safe error handling. Preserve fresh-auth and audit controls on sensitive exports; do not add an uncontrolled export path.
- Accounting business logic expert: calculate results only from POSTED and REVERSED journal entry lines; keep Profit & Loss activity within the selected accounting period and calculate Balance Sheet balances cumulatively through the period end date.
- Enterprise finance and controls expert: expose source provenance, reporting window, currency, ledger balance state, account-classification coverage, and limitations. Keep report values reconcilable to the trial balance and close-assurance evidence.
- OHADA/SYSCOHADA-aware platform architect: preserve account type, mapping key, SYSCOHADA class, country-pack, and certification boundaries. Treat EBITDA as a management measure with a documented
```

formula because it is not a universally defined statutory subtotal. Do not call this suite a certified SYSCOHADA filing.
- SaaS modularity specialist: make the reporting suite discoverable from Accounting, valuable at every period review, tenant-safe, observable, and integrated with the existing Accounting experience rather than implemented as a disconnected analytics dashboard.

Universal operating principles:

- Preserve domain boundaries, service ownership, dependency order, and platform modularity.
- Prefer service-owned read models before UI surfaces.
- Keep business truth server-side; do not create dashboard-only features.
- Enforce tenant isolation, RBAC, module entitlement, fresh auth where relevant, auditability, redaction, and safe error handling.
- Respect release gates, policy gates, boundary gates, service-boundary rules, and existing architecture.
- Keep changes surgical and do not touch unrelated lint warnings, broad refactors, or unrelated modules.
- Inspect relevant reports, code paths, services, actions, components, tests, routes, architecture graphs, and `what-next/` artifacts before implementing.
- Preserve all dirty-worktree changes and never overwrite the in-progress trial-balance improvements.
- Produce evidence, not assumptions.

Tasks:

1. Confirm the gap between canonical accounting reports and the existing operational analytics Financial Summary/Cash Flow models; do not conflate the two.
2. Add a tenant-scoped service-owned read model that resolves a selected accounting period, currency, posted/reversed ledger lines, Profit & Loss sections, management EBITDA reconciliation, and Balance Sheet sections.
3. Use deterministic classification: exact account type and explicit mapping keys first. Never classify from localized account names. Report fallback/unmapped classification coverage and document limitations.
4. For the selected period, calculate Profit & Loss from activity within the period. Calculate the Balance Sheet cumulatively through the selected period end date so opening balances are retained.
5. Add a protected server action using `accounting.reports.read`; validate period input and reject cross-tenant or unavailable periods safely.
6. Build a professional Financial Reports route with a period selector, source/certification banner, KPI summary, Profit & Loss, EBITDA bridge, Balance Sheet, balance check, empty/error states, responsive compact tables, and accessible semantics.
7. Make the suite discoverable from the Accounting overview. Do not modify the already-dirty sidebar unless a conflict-free surgical edit is proven safe.
8. Add focused service/read-model tests covering tenant filters, posted/reversed-only scope, period-vs-cumulative windows, EBITDA classification, fallback coverage, and balance-state calculations.
9. Run focused tests, TypeScript verification, and an authenticated desktop/mobile route smoke. Capture screenshots when the route is renderable.

```
10. Save a concise implementation and verification report under `what-next/`.

Non-goals:

- Do not claim certified OHADA/SYSCOHADA statutory statement status.
- Do not implement a statutory cash-flow statement until explicit cash-account and
operating/investing/financing mapping contracts exist.
- Do not reuse sales-order, product-cost, cash-drawer, or other operational analytics as ledger
statement truth.
- Do not add schema migrations or infer accounting classification from account names.
- Do not add uncontrolled CSV/PDF exports; existing sensitive-export controls must be extended
deliberately in a separate slice.
- Do not touch unrelated lint warnings, broad refactors, unrelated modules, or existing user
changes.
- Do not make database-destructive changes.

Success criteria:

- Authorized users can open one Accounting Financial Reports route and select a tenant-owned
accounting period.
- The page shows a posted-ledger Profit & Loss, a transparent management EBITDA
formula/reconciliation, and a cumulative Balance Sheet.
- EBITDA is visibly marked as a management measure and includes classification coverage/limitations;
the suite is visibly marked as an internal report, not a certified OHADA filing.
- All reads are organization-scoped, period ownership is verified server-side, and
`accounting.reports.read` is enforced at both route and action boundaries.
- Report values are deterministic, currency-consistent, and use only POSTED/REVERSED journal
entries.
- The Balance Sheet exposes assets, liabilities, equity/current earnings, difference, and
balanced/out-of-balance state.
- The UI is compact, responsive, keyboard-readable, has useful empty/error states, and is
discoverable from Accounting.
- Focused tests and TypeScript checks pass, authenticated route smoke is recorded, and no unrelated
dirty-worktree changes are overwritten.
```

Execution Checklist

1. Discovery
    - Compare canonical accounting reports with operational analytics reports.
    - Inspect recent `what-next/` context and `graphify-out/` service/action/route nodes.
2. Existing Implementation Review
    - Inspect the ledger report service/action, accounting periods, chart-of-account metadata, permissions, Accounting shell components, route tests, and sidebar/overview discovery paths.

3. Gap Analysis
    - Separate ledger-supported statements from items requiring new statutory mappings, certification, exports, or cash-flow classification.
4. Implementation
    - Implement service/read model first, protected action second, UI third, and focused tests/evidence last.
5. Verification
    - Record each focused test, typecheck, browser smoke, and any skipped or blocked release gate.

Evidence To Inspect

- `services/accounting/reports.service.ts`
- `services/accounting/periods.service.ts`
- `services/accounting/accounting-settings.service.ts`
- `actions/accounting/reports.actions.ts`
- `services/analytics/financial-reports.service.ts`
- `app/[locale]/(dashboard)/dashboard/accounting/`
- `app/[locale]/(dashboard)/dashboard/analytics/reports/`
- `config/sidebar.ts`
- `prisma/schema.prisma`
- `graphify-out/graph.json`
- relevant focused tests and `what-next/` artifacts
- OHADA AUDCIF/SYSCOHADA official publication and IFRS Foundation IFRS 18 EBITDA materials

Expected Artifacts

- This refined execution prompt.
- Service-owned financial statement read model and protected action.
- Professional Accounting Financial Reports route and focused navigation change.
- Focused unit/route tests.
- Authenticated desktop/mobile screenshots or a documented browser blocker.
- Final implementation report under `what-next/`.

Verification Commands

```
npx jest --runInBand <focused financial-reporting tests>
npm run typecheck
```

```
npm run policy:gates
```

Use the narrowest available route/browser smoke. Run `npm run build:app` only if route rendering or release-readiness scope requires it and the existing workspace state can support it safely.

Risk Controls

- Tenant isolation: every account, period, entry, line, settings, and organization lookup must include the organization boundary.
- RBAC/module entitlement: route and action use the existing Accounting permission and module enforcement path.
- Accounting truth: only posted/reversed ledger lines; no sales-order or cash-drawer substitution.
- Period semantics: Profit & Loss is period activity; Balance Sheet is cumulative through period end.
- EBITDA: explicit formula, explicit mappings, coverage disclosure, and management-measure label.
- Certification: internal report only; never imply OHADA certification or Close & Assurance signature.
- Redaction: no contact, authentication, payroll-person, provider-secret, or unrelated audit fields.
- Dirty worktree: preserve the in-progress Trial Balance and the pre-existing sidebar changes.
- Release gates: document focused results and avoid claiming full readiness from partial checks.

Optional Next Prompts

1. Extend the secured accounting-export pipeline with Financial Statements CSV/PDF exports, watermarking, fresh authentication, and evidence hashes.
2. Design expert-reviewed SYSCOHADA statement-line mappings and country-pack provenance for certified statutory presentation.
3. Add a true ledger cash-flow statement after introducing explicit cash-account and operating/investing/financing classification contracts.